

ML Key Concepts

Data: Features

Features: Measurable data that helps make predictions

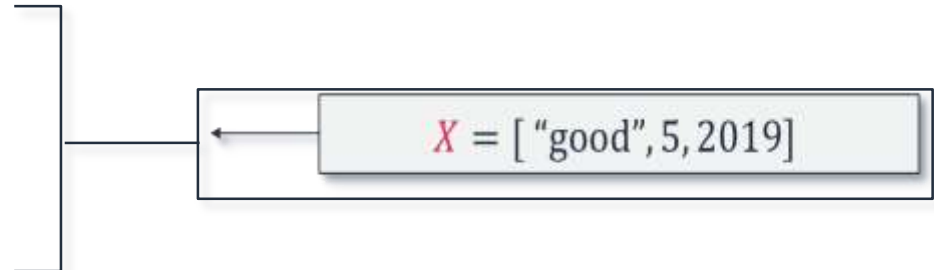
- Single feature: x
- Collection of features: Vector X

Example book feature: Year sold



Example set of book features:

- Book condition
- Month sold
- Year sold



Data: Label

Label: The output that you want to predict

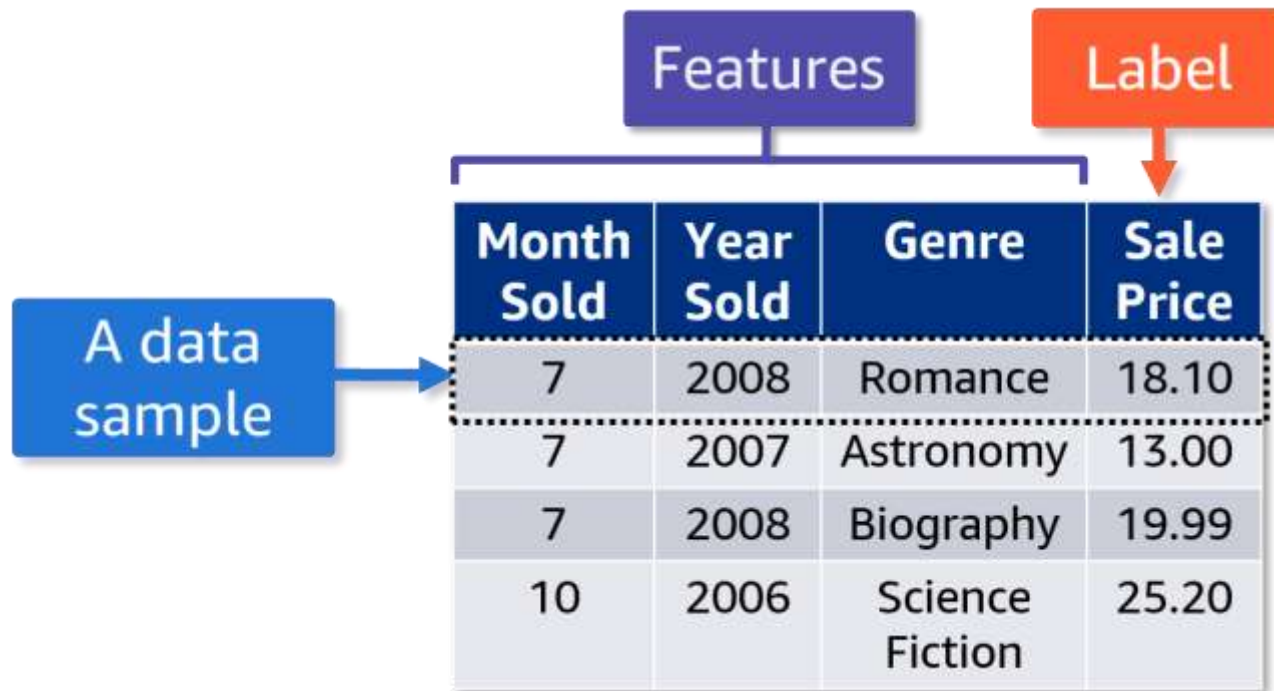
- Sometimes called the *target*
- For a single data sample, usually denoted with the variable y

A particular book target: **book sale price**



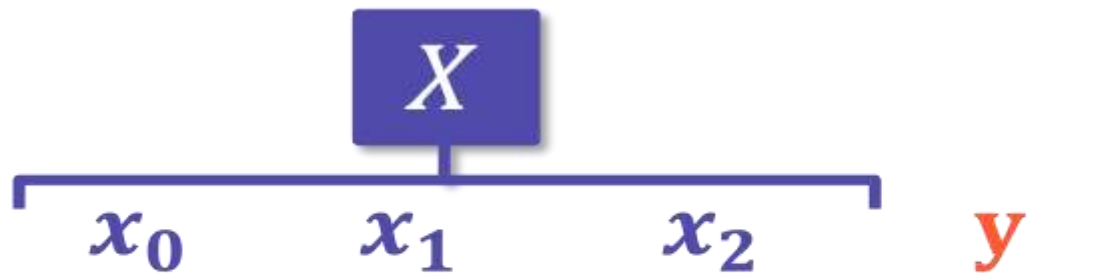
Tabular data: Features and labels

- Rows represent each data **sample**.
- Columns represent each **feature and the label**.



Tabular data: Vectors and variables

- Features are represented by a vector X .
- Labels are represented by a variable y .



A diagram above the table shows a purple box with the letter X inside. A line extends from the bottom of this box, branching into three lines that point to the column headers x_0 , x_1 , and x_2 . To the right of this, a red letter y is positioned above the 'Sale Price' column header.

x_0	x_1	x_2	y
Month Sold	Year Sold	Genre	Sale Price
7	2008	Romance	18.10
7	2007	Astronomy	13.00
7	2008	Biography	19.99
10	2006	Science Fiction	25.20

ML model: An example

The goal is to predict a good suggested book price.

prediction $\sim$ weighted combination of **features**

The diagram shows the equation $\hat{y} = f(x) \approx a_0x_0 + a_1x_1 + \dots + a_4x_4 + \dots + a_nx_n$. A green bracket is above $\hat{y}$ and a blue bracket is above the entire right-hand side. Below the equation, a horizontal line with four upward-pointing arrows connects to a dark blue box containing the text "(Year sold, page count, genre, additional fields)".

$$\hat{y} = f(x) \approx a_0x_0 + a_1x_1 + \dots + a_4x_4 + \dots + a_nx_n$$

Suggested
price

(Year sold, page count, genre, additional fields)

ML model: Feature 1

- x_1 : Number of pages for a given book

prediction $\sim$ weighted combination of **features**



The equation $\hat{y} = f(x) \approx a_0x_0 + a_1 * 150 + \dots + a_4x_4 + \dots + a_nx_n$ is shown. A green bracket is under $\hat{y}$. A blue bracket is under the entire right-hand side of the equation. A blue box with a purple arrow points from the value 150 to the text $x_1 = 150$.

$$\hat{y} = f(x) \approx a_0x_0 + a_1 * 150 + \dots + a_4x_4 + \dots + a_nx_n$$

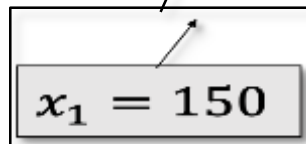
$$x_1 = 150$$

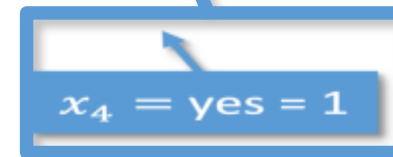
ML model: Feature 4

- x_1 : Number of pages for a given book
- x_4 : Is this book fiction?

prediction $\sim$ weighted combination of **features**

$$\hat{y} = f(x) \approx a_0 x_0 + a_1 * 150 + \dots + a_4 * 1 + \dots + a_n x_n$$


$$x_1 = 150$$

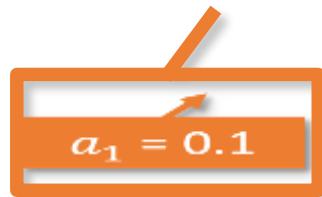

$$x_4 = \text{yes} = 1$$

ML model: Weight 1

• **Weights**, in this model, determine how much influence a given feature has on the output (**prediction**)

- a_1 : Weight for “Number of pages” feature x_1

$$\hat{y} = f(x) \approx a_0 x_0 + 0.1 * 150 + \dots + a_4 * 1 + \dots + a_n x_n$$



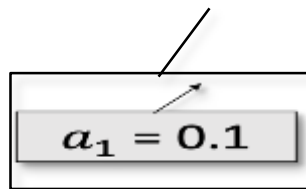
$a_1 = 0.1$

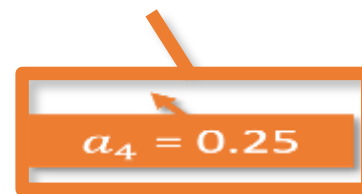
ML model: Weight 4

• **Weights**, in this model, determine how much influence a given feature has on the output (**prediction**)

- a_1 : Weight for “Number of pages” feature x_1
- a_4 : Weight for “Is fiction” feature x_4

$$\hat{y} = f(x) \approx a_0 x_0 + 0.1 * 150 + \dots + 0.25 * 1 + \dots + a_n x_n$$


$$a_1 = 0.1$$


$$a_4 = 0.25$$

ML model: Prediction

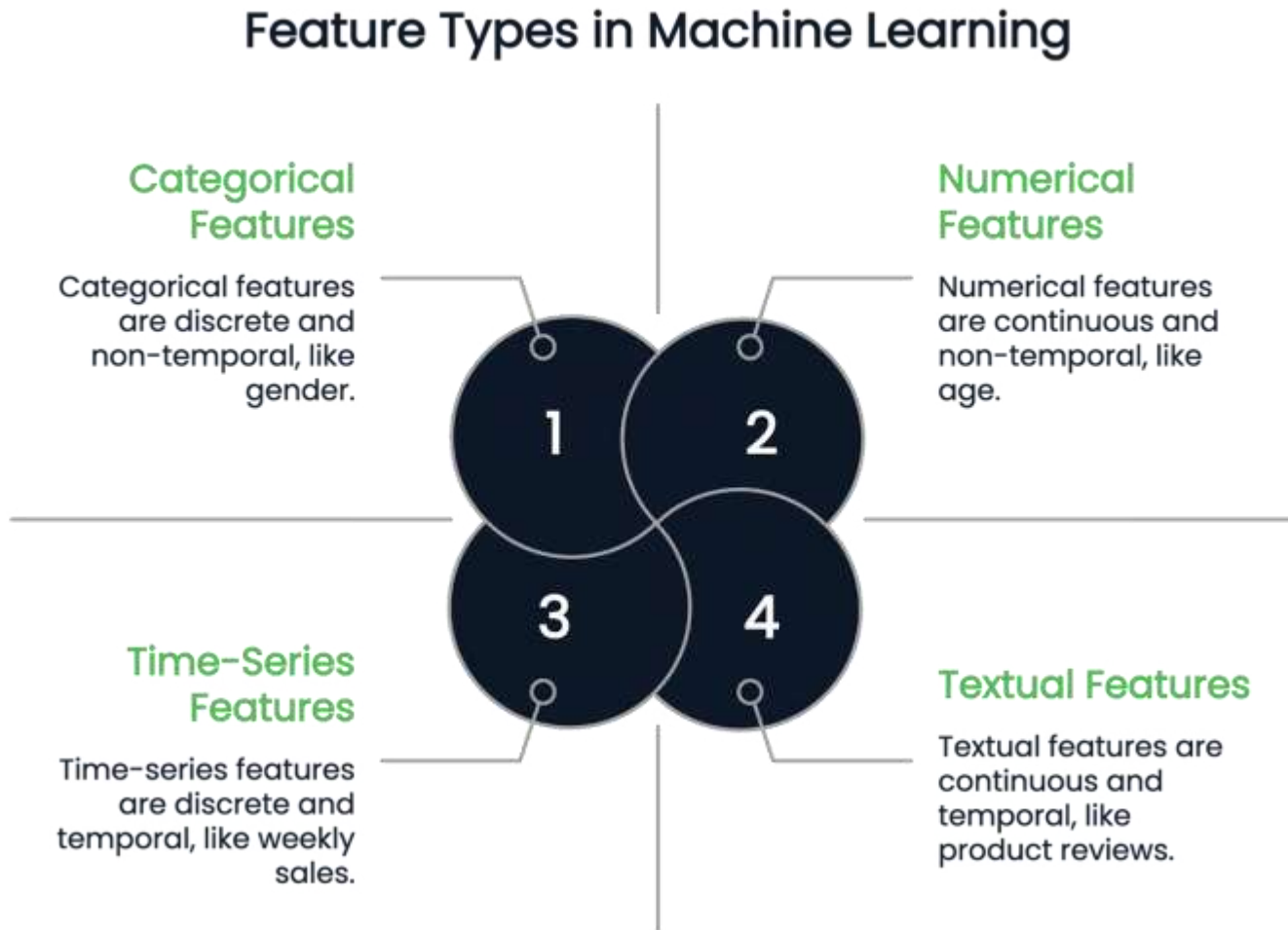
For this model, use these two features to determine a **price to suggest**.

$$\hat{y} = f(x) \approx 0.1 * 150 + 0.25 * 1 = 15.25$$

Suggested price of a 150-page book in the fiction genre is \$15.25

Note: This example shows how a model can make predictions; the method to learn these weights from data will be explained later.

Types of Features in Machine Learning



Types of Features in Machine Learning

1. Numerical features

- Numerical features represent data in numbers.
- They are continuous quantitative variables.
- Examples include height, age, and salary.

1. Categorical features

- A categorical column can contain only discrete features.
- For example, a person's gender is a categorical column.
- Categorical variables are further divided into: binary and non-binary types.
 - Binary variables have two possible categories
 - Non-binary features can have multiple categories

Types of Features in Machine Learning



3. Textual

- Textual columns contain only text data.
 - Examples : product reviews or product description columns in a retail dataset.

4. Time-series features

- Time-series features represent timely data
 - Example: Weekly sales or stock price fluctuations over a year.

Machine Learning Pipeline

Step 1: Data Collection and Preprocessing

Step 2: Feature Engineering

Step 3: Data splitting

Step 4: Model Selection & Training

Step 5: Model evaluation & Optimization

Step 6: Model Deployment

Step 7: Continuous learning & Monitoring

Defining the problem

Define the objective:

A specific outcome that you want to use ML to achieve.

For example, to help someone achieve something or some business goal.

Increase revenue by ensuring all products in catalog have prices & can be surfaced to customer.

Translate the objective to an ML problem:

Express the objective in ML terms; for example, predicting or calculating the probability.

Predict the price (continuous numerical value) for all products in the catalog where the price is missing.

-Step 1: Data Collection and Preprocessing

1. The first step is to collect data from relevant sources, which might include **databases, CSV files, APIs, or web scraping**. The quality and quantity of the data you collect will significantly impact the performance of your machine learning model.
2. After collecting the data, **understand** the features (columns), the target variable (what you want to predict), the types of data (numeric, categorical, text), and identify potential issues such as missing values, outliers, or inconsistencies.
3. **Data preprocessing** is the process of transforming raw data into a clean and usable format.

■ Step 2: Feature Engineering

Feature Engineering is the **process of selecting, creating or modifying features** like input variables or data to help machine learning models learn patterns more effectively.

It involves transforming raw data into meaningful inputs that improve model accuracy and performance.

-Step 3: Data splitting

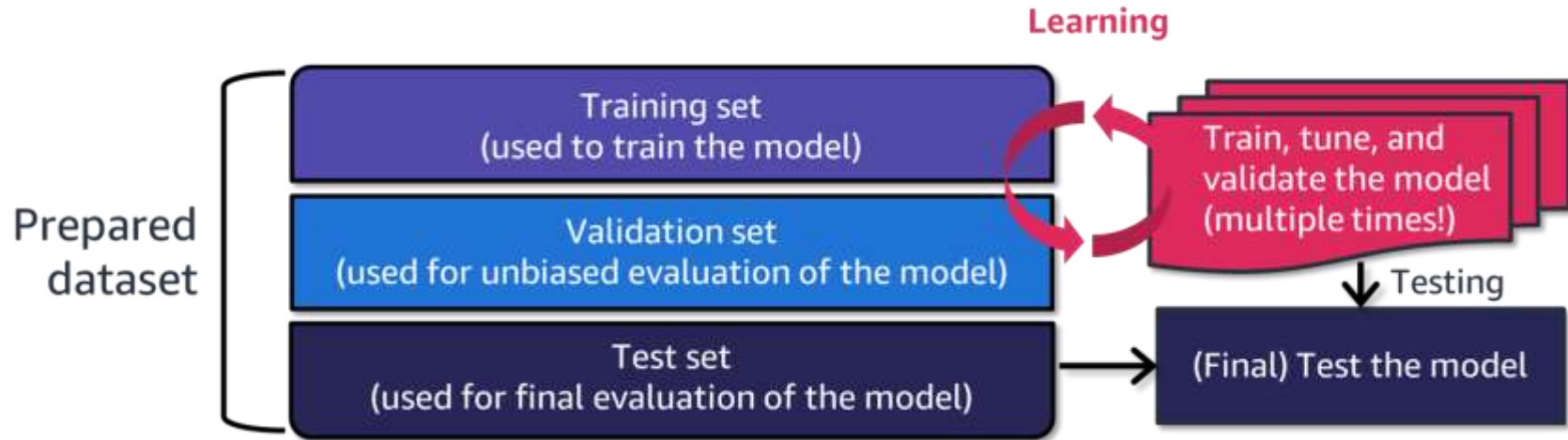
Train-Test Split:

- **Split Data:** To evaluate the performance of a model, the dataset is typically split into a training set (used to train the model) and a testing set (used to evaluate the model's performance). A common split ratio is 70–30 or 80–20, depending on the size of the dataset.
- **Why Split Data?:** Splitting the data helps prevent overfitting, where a model performs well on training data but poorly on unseen data. By testing the model on a separate set of data (the test set), you can get a better estimate of its real-world performance.

Step 4: Model Selection & Training

- **Supervised models:** Train (learn) by comparing the error between predictions and true values (labels).
- **Unsupervised models:** Train (learn) by comparing the results of grouping similar things together. Similarity can be defined in different ways; for example, distance or intersections of features.

Step 5: Model evaluation & Optimization



—Step 6: Model Deployment

Model Deployment is the process of taking a trained machine learning model and making it available in a real-world environment, so it can receive new data and make predictions for users or applications.

—Step 7: Continuous learning & Monitoring

Continuous Learning: Retrain or update the model periodically using new data/feedback to keep it accurate as real-world patterns change.

Monitoring: Continuously track model performance (accuracy, drift in data/predictions) on live data to detect when the model starts degrading.

Data processing: Preparing the data

- Data preprocessing is the process of transforming raw data into a clean and usable format.
- This involves a series of tasks aimed at **improving the quality of the data**, making it more suitable for machine learning algorithms.
- Convert all values to numbers.

Sale Price	Type	Description	imgID
6.99	Grocery	Sugar Free ...	51Q696oAVL
26.39	Health	Sports Recovery ...	31WGV435LL
9.79	Grocery	Simple Jam...	41c3R441GfL
49	Beauty	Therapy Oil ...	51Mv1232QK

Sale Price	Type	Description	imgID
6.99	26	424	1068
26.39	40	863	284
9.79	26	1039	653
49	51	492	942

Tables 1 and 2: Example of conversion from an original dataset to only numerical values

Why Preprocess the Data?

Noisy: Containing errors or outliers.

E.g., Salary = “-10” Noisy data may come from

- Human or computer error at data entry.
- Errors in data transmission.

Missing: lacking certain attribute values or containing only aggregate data. **E.g., Occupation = “”**. Missing (Incomplete) data may come from

- “Not applicable” data value when collected.
- Human/hardware/software problems.

Inconsistent: Data inconsistency meaning is that different versions of the same data appear in different places. Inconsistent data may come from

- Errors in data entry.
- Merging data from different sources with varying formats.
- Differences in the data collection process.

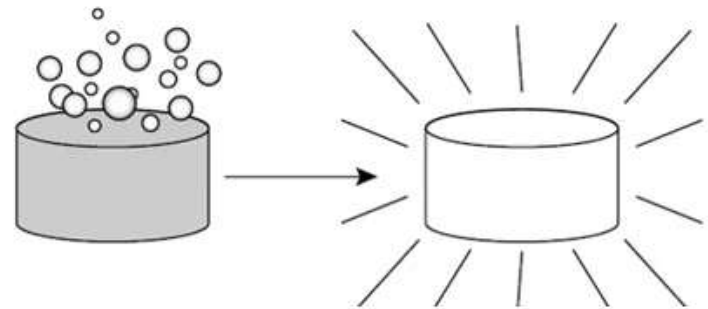
Steps in Data Preprocessing

1. Data Cleaning

Data cleaning is the process of **identifying and correcting errors or inconsistencies** in the data to ensure it is accurate and complete. The objective is to address issues that can distort analysis or model performance.

For example:

- **Handling missing values:** Using strategies like mean/mode imputation, deletion, or predictive models to fill in or remove missing data.
- **Removing duplicates:** Eliminating duplicate records to ensure each entry is unique and relevant.
- **Correcting inconsistent formats:** Standardizing formats (e.g., date formats, string cases) to maintain consistency.



Missing Values

Handling missing data : Missing data can negatively impact the performance of a machine learning model or analysis. There are several strategies to handle missing values effectively:

- **Imputation:** This technique involves filling in missing values with a calculated estimate, such as the mean, median, or mode of the available data. Advanced methods include predictive modeling, where missing values are predicted based on relationships within the data.
- **Deletion:** Removing rows or columns with missing values is a straightforward solution. However, it should be used cautiously as it can lead to loss of valuable data, especially if many entries are missing.
- **Modeling missing values:** In cases where the missing data pattern is more complex, machine learning models can predict the missing values based on the rest of the dataset. This can improve accuracy by incorporating relationships between different variables.

Impute Missing Data:

- **Mean/Median Imputation:** Replace missing values with the mean or median of the column, which is useful for numerical data.
- **Mode Imputation:** For categorical data, replace missing values with the most frequent value (mode).
- **Predictive Imputation:** Use models like k-Nearest Neighbors (KNN) to predict and fill in missing values based on other available data.
- **Forward/Backward Fill:** In time series data, missing values can be filled based on previous (forward fill) or subsequent (backward fill) values.

Duplicate Data Removal:

Identify Duplicates: Duplicates can occur due to data entry errors or merging datasets. They can be identified using functions like `pandas.DataFrame.duplicated()` in Python.

Remove Duplicates: Once identified, duplicates should be removed to prevent them from skewing the analysis or model results.

Outlier detection and removal

Outliers are extreme values that deviate significantly from the rest of the data, which, like missing values, can distort analysis and model performance. Various techniques can be used to detect and handle outliers:

- **Z-Score method:** This approach measures how many standard deviations a data point is from the mean. Data points beyond a certain threshold (e.g., ± 3 standard deviations) can be considered outliers.
- **Interquartile range (IQR):** IQR is the range between the first quartile (Q1) and the third quartile (Q3). Values beyond 1.5 times the IQR above Q3 or below Q1 are considered outliers.
- **Visual techniques:** Visualization methods like box plots, scatter plots, or histograms can help detect outliers in a dataset. Once identified, outliers can either be removed or transformed, depending on their influence on the analysis.

Noisy Data

Noise is a random error or variance in a measured variable.

Data smoothing techniques are used to eliminate noise and extract the useful patterns.

Noisy Data- The different techniques used for data smoothing are:

1.Binning: Binning methods smooth a sorted data value by consulting its “neighbourhood,” that is, the values around it. The sorted values are distributed into several “buckets,” or bins.

There are three kinds of binning. They are:

- **Smoothing by Bin Means:**In this method, each value in a bin is replaced by the mean value of the bin.
- **Smoothing by Bin Medians:**In this method, each value in a bin is replaced by the median value of the bin.
- **Smoothing by Bin Boundaries:**In this method, the minimum and maximum values in each bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value.

Example:

Sorted data for price (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15

Bin 2: 21, 21, 24

Bin 3: 25, 28, 34

Smoothing by bin means:

Bin 1: 9, 9, 9

Bin 2: 22, 22, 22

Bin 3: 29, 29, 29

Smoothing by bin medians:

Bin 1: 8, 8, 8

Bin 2: 21, 21, 21

Bin 3: 28, 28, 28

Smoothing by bin boundaries:

Bin 1: 4, 4, 15

Bin 2: 21, 21, 24

Bin 3: 25, 25, 34

2. Regression: Data Smoothing can also be done by regression, a technique that used to predict the numeric values in a given data set. It **analyses the relationship between a target variable (dependent) and its predictor variable (independent)**.

- Regression is a form of a supervised machine learning technique that tries to predict any continuous valued attribute.

3. Clustering: It supports in identifying the outliers. The similar values are organized into clusters and those values which fall outside the cluster are known as outliers.

Data encoding

When working with categorical data, encoding is necessary to convert categories into numerical representations that machine learning algorithms can process.

Encoding Techniques:

- **Label Encoding:**

- Label encoding converts **categorical variables into integers**.
- For example, the categories “red”, “green”, “blue” could be encoded as 0, 1, and 2, respectively. This method is simple but can introduce ordinal relationships where none exist.

- **One-Hot Encoding:**

- One-hot encoding converts **categorical variables into a binary matrix**, where each category is represented by a binary vector.
- For example, “red”, “green”, “blue” would become three binary columns, with only one column being 1 for each category and the others being 0.

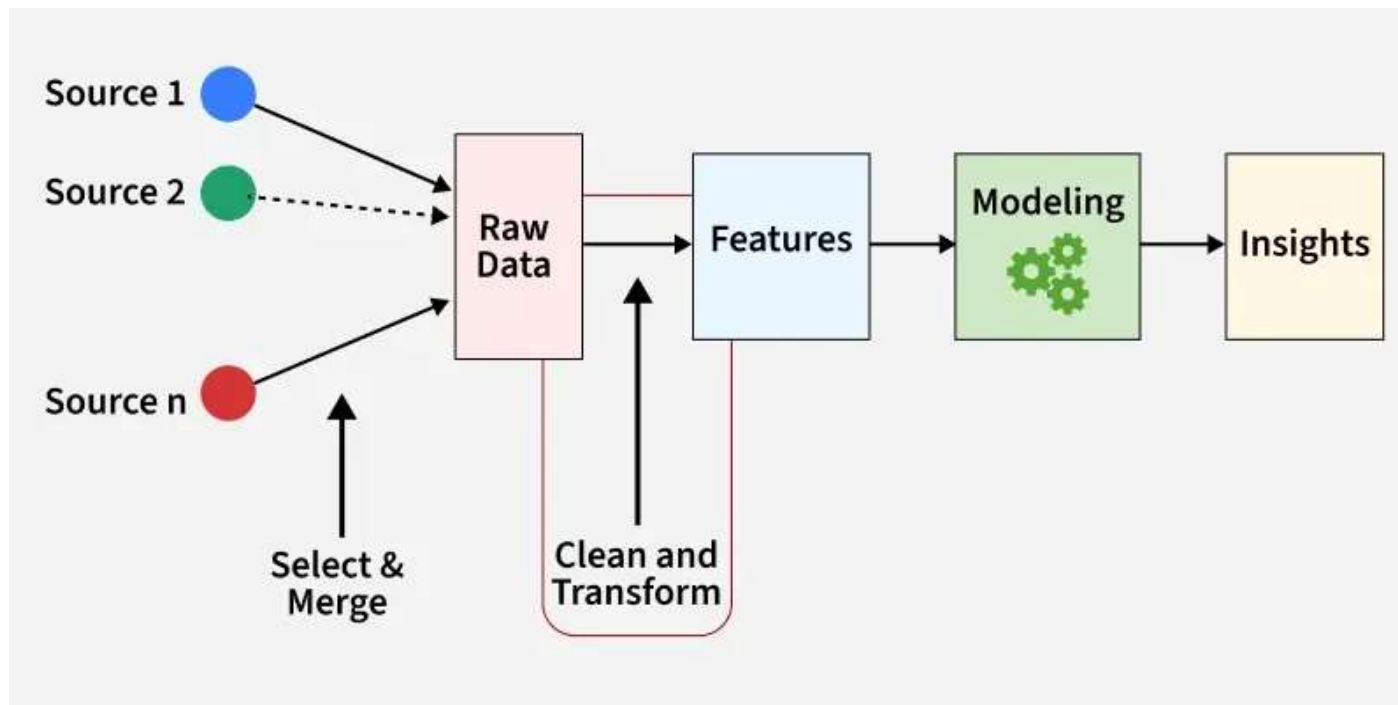
Data scaling and normalization

Scaling and normalization ensure that numerical features are on a similar scale.

- **Min-max scaling:** This technique scales data to a specified range, typically 0 to 1. It's useful when all features need to have the same scale.
- **Normalization:** Normalization scales the data to a range between 0 and 1 or -1 and 1. This is particularly useful when the model relies on distance metrics, like K-Nearest Neighbors (KNN) or Neural Networks.
- **Standardization:** Standardization rescales data so that it has a mean of 0 and a standard deviation of 1, which is important for algorithms like Support Vector Machines (SVM) or logistic regression.

FEATURE ENGINEERING

- Feature Engineering is the process of **selecting, extracting, and transforming the most relevant features** from the available data to build more accurate and efficient machine learning models.



Importance of Feature Engineering

- **Improve accuracy:** Choosing the right features helps the model learn better, leading to more accurate predictions.
- **Reduce overfitting:** Using fewer, more important features helps the model avoid memorizing the data and perform better on new data.
- **Boost interpretability:** Well-chosen features make it easier to understand how the model makes its predictions.
- **Enhance efficiency:** Focusing on key features speeds up the model's training and prediction process, saving time and resources.

FEATURE ENGINEERING



- Feature Creation
- Feature Transformation & Scaling
- Feature Extraction
- Feature Selection

1. Feature Creation:



Feature creation involves generating new features from domain knowledge or by observing patterns in the data. It can be:

- **Domain-specific:** Created based on industry knowledge like business rules.
- **Data-driven:** Derived by recognizing patterns in data.
- **Synthetic:** Formed by combining existing features.

2. Feature Scaling & Transformation

Transformation adjusts features to improve model learning:

- **Normalization & Scaling:** Adjust the range of features for consistency.
- **Encoding:** Converts categorical data to numerical form i.e one-hot encoding.
- **Mathematical transformations:** Like logarithmic transformations for skewed data.

Feature scaling



Feature scaling ensures that numerical features lie within a standardized range, preventing some features from dominating the learning process due to their larger values.

Eg: datasets in which different columns have different units – like one column can be in kilograms, while another column can be in centimeters.

Eg: we can have columns like income which can range from 20,000 to 100,000, while an age column which can range from 0 to 100.

When we feed these features to the model as is, there is every chance that the income will influence the result more due to its larger value.

But this doesn't necessarily mean it is more important as a predictor. So, to give importance to both Age, and Income, we need feature scaling.

1. Normalization (min-max scaling):

This method scales all feature values to fall within 0 and 1. It subtracts the minimum value of the column from each data point and then divides it by the range of that column, which is the difference between the maximum and minimum values.

Scaled value = (datapoint - min(column)) / (max(column) - min(column))

1. Standardization (Z-score scaling):

This transforms all features to have a mean of 0 and a standard deviation of 1. The formula is: the mean of a column is subtracted from each data point of that column, and the residue is divided by the standard deviation of that feature.

Scaled value = (datapoint - mean(column)) / std(column)

Feature Extraction

- In feature extraction new features are created from a combination of original features.
- Some of the commonly used operators for combining the original features include:
 1. For Boolean features: Conjunctions, Disjunctions etc.
 2. For nominal features: Cartesian product etc.
 3. For numerical features: Min, Max, addition etc.

Feature Extraction

Feat A	Feat B	Feat C	Feat D
34	34.5	23	233
44	45,56	11	3.44
78	22.45	21	4.5
22	65.22	11	322
11	122.3 5	63	23.2



Feat 1	Feat 2
41.25	185.80
54.20	53.12
43.73	35.79
37.02	239.42
113.39	167.74

3. Feature Extraction:



- **Dimensionality reduction:** Techniques like PCA reduce features while preserving important information.
- **Aggregation & Combination:** Summing or averaging features to simplify the model.

Importance of Feature Extraction



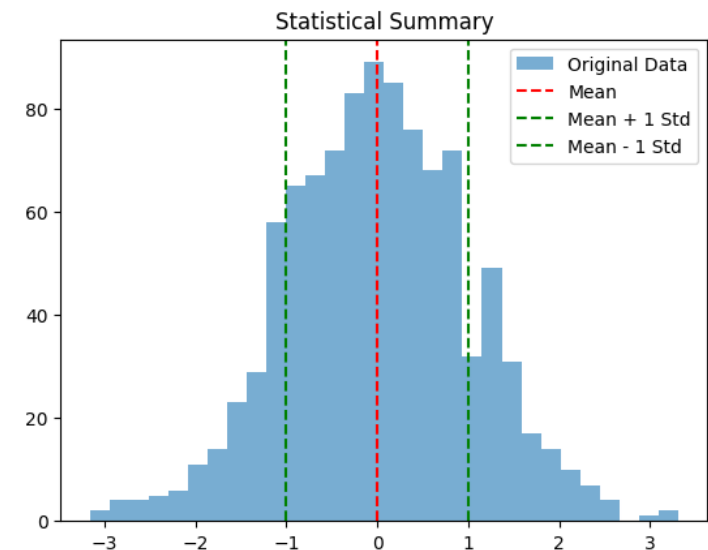
- Reduces computation by simplifying complex raw data.
- Improves model performance using relevant features.
- Provides better insights by removing noise.
- Helps prevent overfitting by reducing feature complexity.

Key Techniques for Feature Extraction

1. Statistical Methods

- **Mean:** The average value of a dataset.
- **Median:** The middle value when it is sorted in ascending order.
- **Standard Deviation:** A measure of the spread or dispersion of a sample.
- **Correlation and Covariance:** Measures of the linear relationship between two or more factors.

These statistical methods can be used to represent the center trend, spread and links within a collection.



2. Dimensionality Reduction

Dimensionality reduction reduces the number of features without losing important information. Some popular methods are:

- **Principal Component Analysis:** It transforms original features into new orthogonal components that capture maximum variance in the data.
- **Linear Discriminant Analysis (LDA):** It finds the best combination of features to separate different classes, maximizing class separability for better classification.

3. Feature Extraction for Textual Data

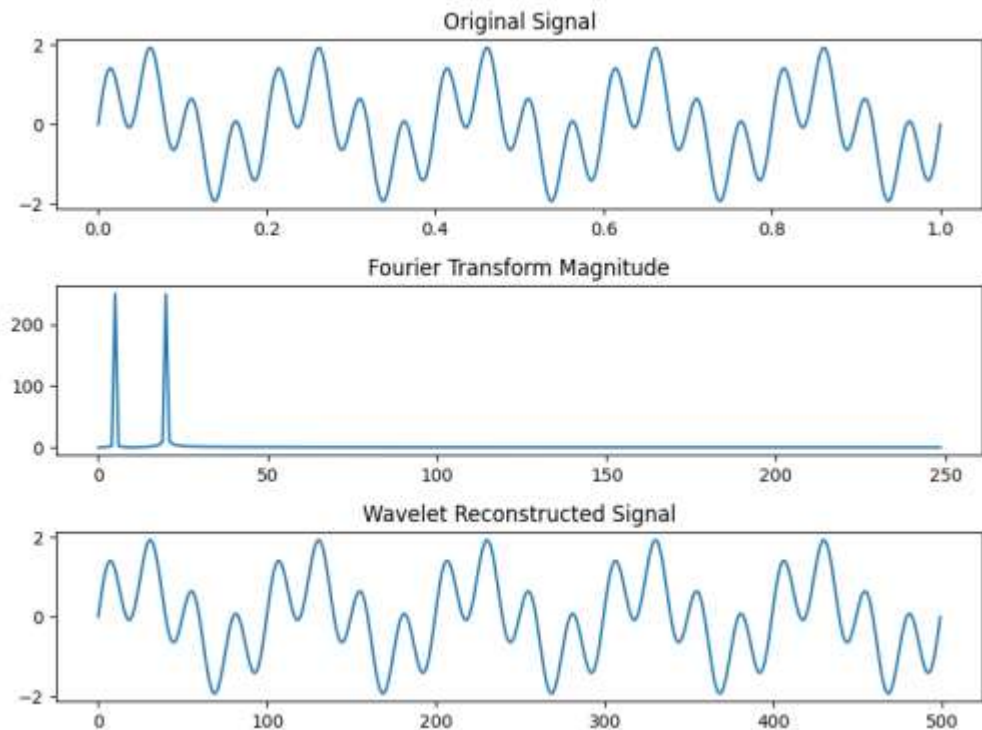
In Natural Language Processing (NLP), we often convert raw text into a format that machine learning models can understand.

1. **Bag of Words (BoW)**: Represents a document by **counting word frequencies**, ignoring word order, useful for basic text classification.
2. **Term Frequency-Inverse Document Frequency (TF-IDF)**: Adjusts word importance based on **frequency in a specific document compared to all documents**, highlighting unique terms.

4. Signal Processing Methods

It is used for analyzing time-series, audio and sensor data:

1. **Fourier Transform:** It converts a signal from the time domain to the frequency domain to analyze its frequency components.
2. **Wavelet Transform:** It analyzes signals that vary over time, offering both time and frequency information for non-stationary signals.



5. Image Data Extraction

1. **Histogram of Oriented Gradients (HOG):** This technique finds the distribution of intensity gradients or edge directions in an image. It's used in object detection and recognition tasks.
2. **Convolutional Neural Networks (CNN) Features:** They learn hierarchical features from images through layers of convolutions, ideal for classification and detection tasks.



Advantages



- **Simplifies Data:** Reduces complex data into a manageable form for easier analysis and visualization.
- **Boosts Model Performance:** Removes irrelevant data, making algorithms faster and more accurate.
- **Highlights Key Patterns:** Filters out noise to focus on important features for quicker insights.
- **Improves Generalization:** Helps models perform better on new, unseen data by emphasizing informative features.
- **Speeds Up Training and Prediction:** Fewer features mean faster model training and real-time predictions.

Challenges

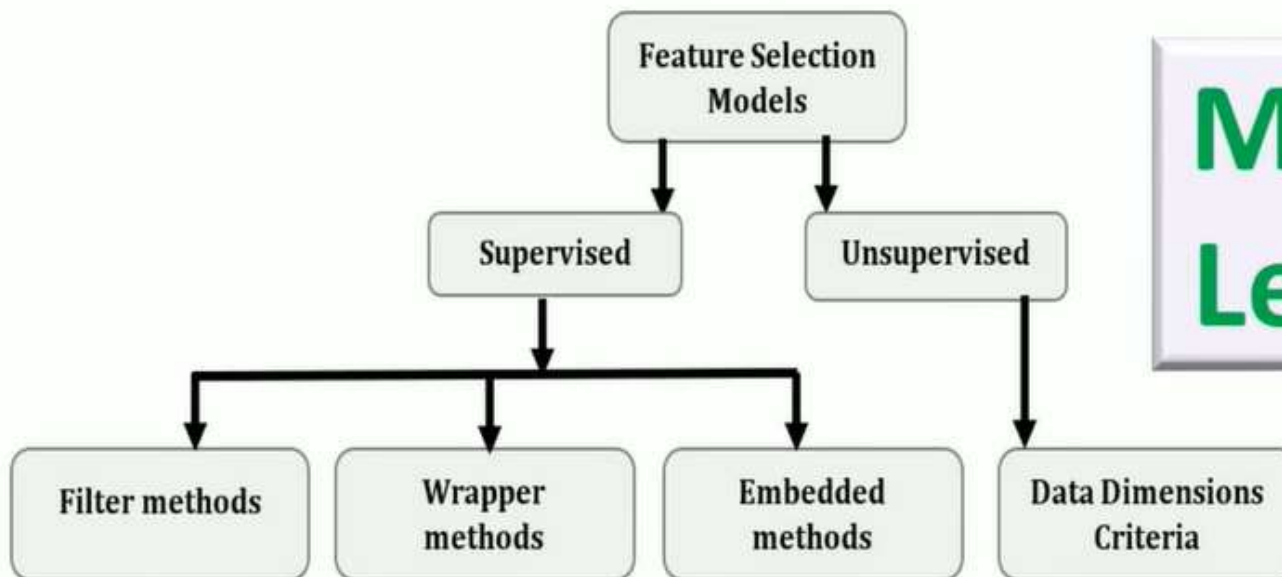
- **Managing High-Dimensional Data:** Extracting relevant features from large, complex datasets can be difficult.
- **Risk of Overfitting or Underfitting:** Too many or too few features can hurt model accuracy and generalization.
- **Computational Costs:** Complex methods may require heavy resources, limiting use with big or real-time data.
- **Redundant or Irrelevant Features:** Overlapping or noisy features can confuse models and reduce efficiency.

4. Feature Selection:



Feature selection involves selecting the most relevant features and removing redundant or irrelevant ones, which helps reduce overfitting and improve efficiency.

Feature Selection



**Machine
Learning**

Feature Selection:

- Machine learning (ML) is a subfield of artificial intelligence that allows computers to learn without being explicitly programmed.
- Usually, machine learning model is built with help of dataset.
- A dataset is usually represented in a tabular form: rows, and columns are features.

Address	Number of Rooms	House Age	owner	price

Feature Selection:

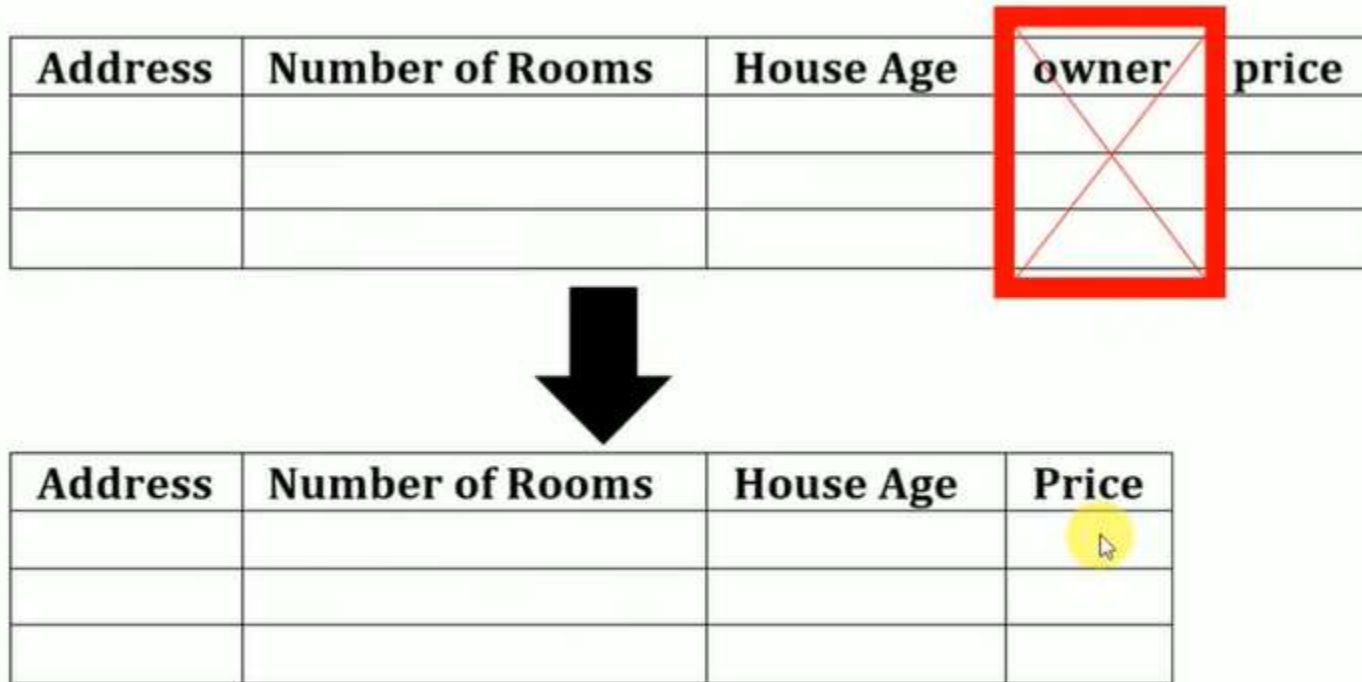


- In machine learning, feature selection selects the most relevant subset of features from the original feature set by dropping redundant, noisy, and irrelevant features.
- Feature selection is used today in many applications like Object detection, NLP, remote sensing, image retrieval, etc.

Why do we need to do feature selection

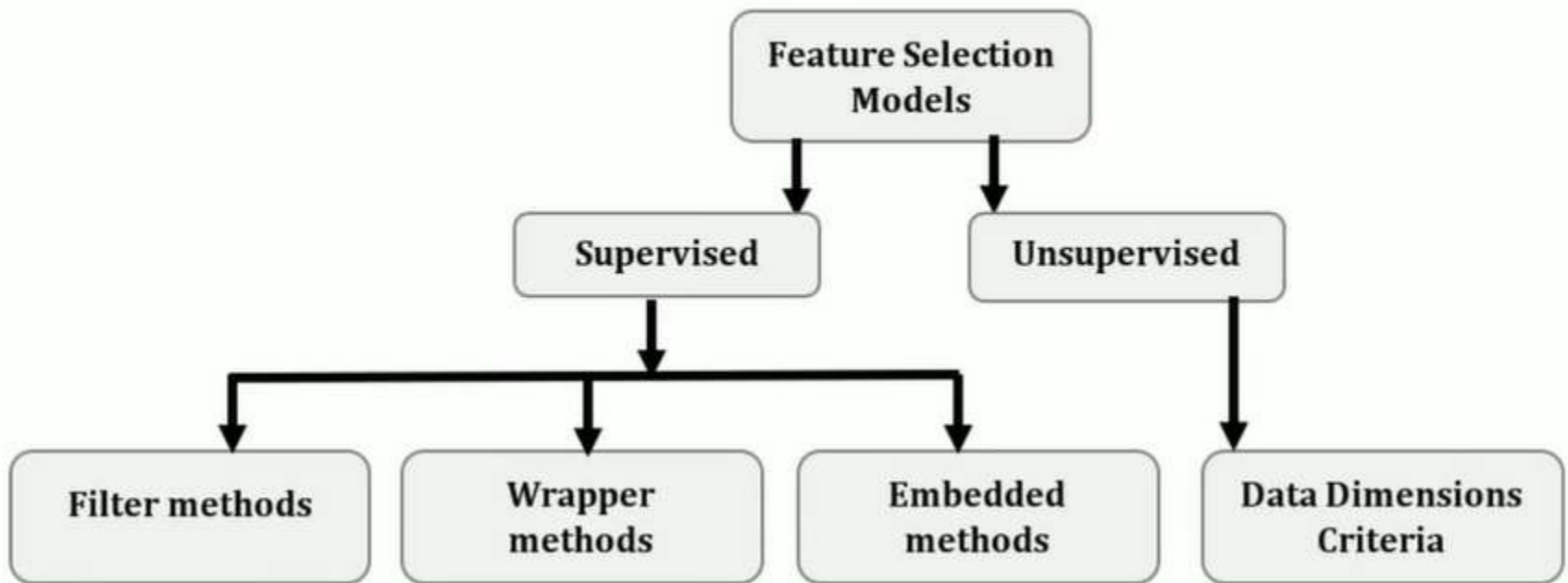
- If there are too many features, our model can may become weak or generate some misleading patterns.
- That happens because, usually, some features aren't correlated with the target variable and represent noise.
- If our model outputs predictions based on such features, its accuracy is likely to be subpar.
- Also, a dataset with many columns slows down the training process.

Why do we need to do feature selection



Feature Selection:

- There are several methods of doing feature selection:



Feature Selection:

- **Unsupervised Methods**

- Unsupervised feature selection methods are applied to unlabeled data.
- An unsupervised selection method **rates** each feature dimension according to a number of factors, including entropy, variance, and the capacity to maintain local similarity.

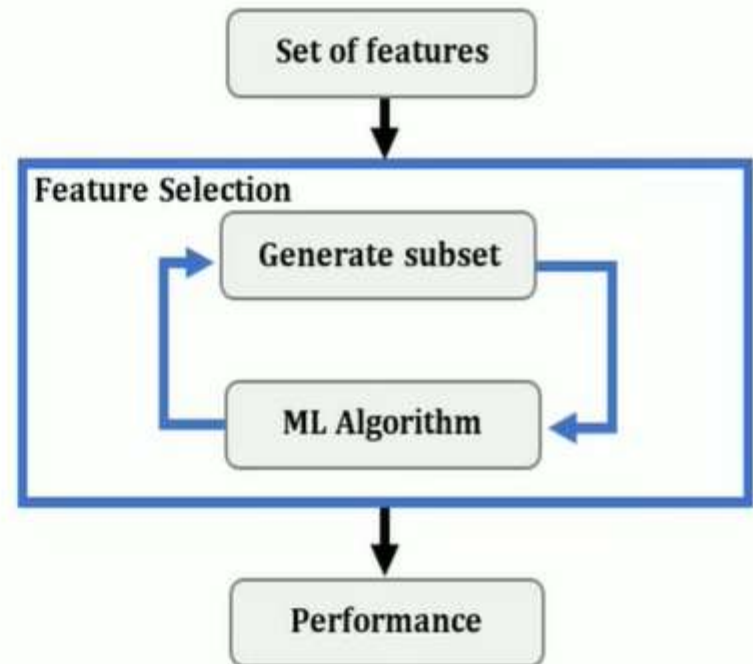
Feature Selection:

- **Supervised Methods**
- On the other hand, we use supervised feature selection methods on labeled data.
- They determine the features that are expected to maximize the supervised model's performance.
- Supervised feature selection methods can be split into three primary categories based on the feature selection strategy.

Subscribe

Wrapper Method

- **Wrapper Methods**
- We use a wrapper method after choosing the ML algorithm to use.
- For each feature subset, we estimate the algorithm's performance by training and evaluating it using only the features in a subset.
- Then, we add or remove features based on the estimate.
- This is an iterative process.



Wrapper Method

We use a greedy strategy to form feature subsets.

- In **forward wrapper methods**, we start from an empty feature set and add the feature maximizing the performance in each step until no substantial improvement is observed.
- So, if there are **n** features, we build **n** ML models in the first iteration.
- Then, we select the feature corresponding to the model with the best performance.
- In the second iteration, we repeat the process with the remaining **n-1** features.
- We continue like this as long as there's a significant performance improvement between models with which we end successive iterations.

Wrapper Method

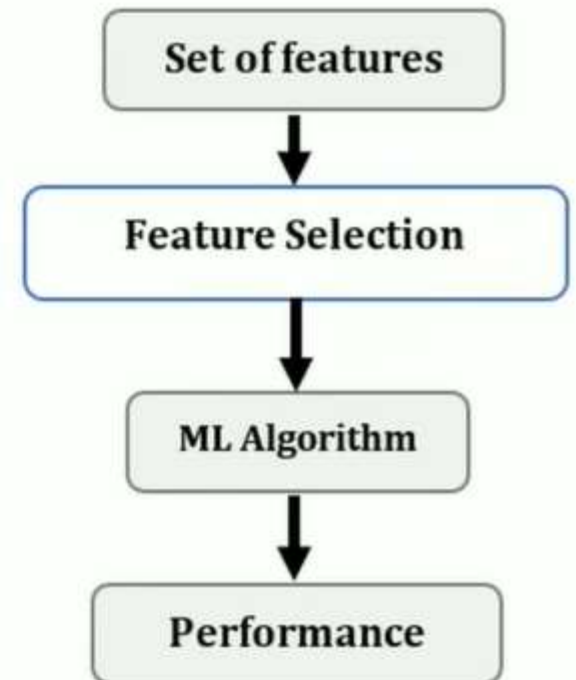
We use a greedy strategy to form feature subsets.

- **Backward methods** work the opposite way.
- They start from the full feature set and remove them one by one.
- Finally, stepwise methods reconsider features.
- So, in each iteration, they can remove a feature previously added as well as add a feature discarded in a previous step.

Filter Methods

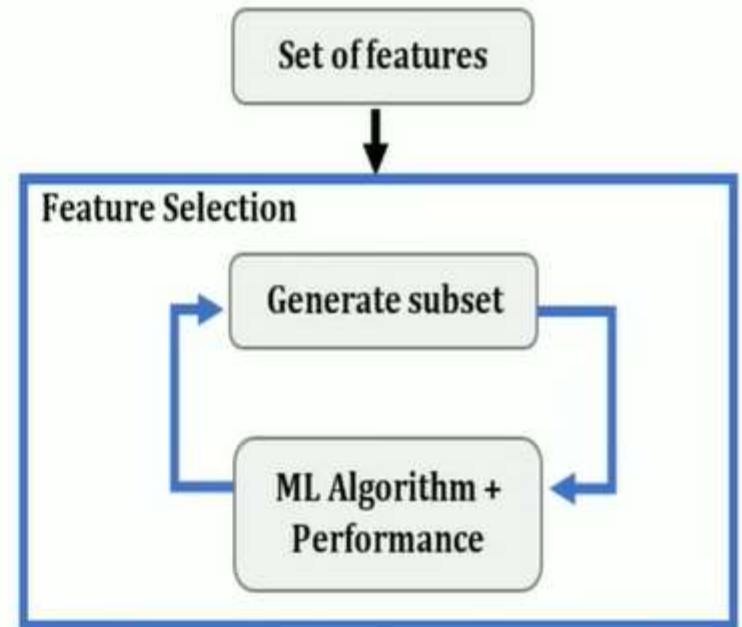
Filter Methods

- Filter methods use statistical tools to select feature subsets based on their relationship with the target.
- These methods remove features with low correlation with the target variable before training the final ML model.
- In doing so, they compute correlation and estimate the strength of the relationship using the, and other statistical tools.
- Chi-Square Test, Information Gain, Fisher's Score, Pearson correlation, ANOVA, variance thresholding



Embedded Methods

- **Intrinsic (or Embedded) Methods**
- Here selection of feature happens simultaneously with and is performed implicitly by the ML algorithm of our choice.
- During training, some steps of the ML algorithm do feature selection:
- For instance, this is the case with decision trees.
- At each node split, they choose the best feature to split the data by.
- Those choices represent feature selection.



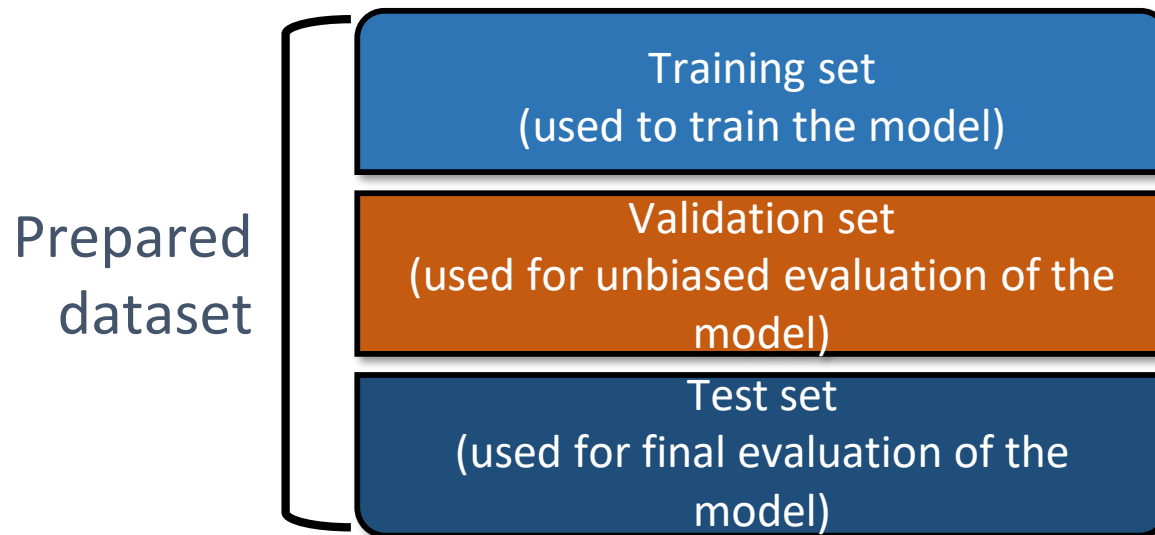
Advantages

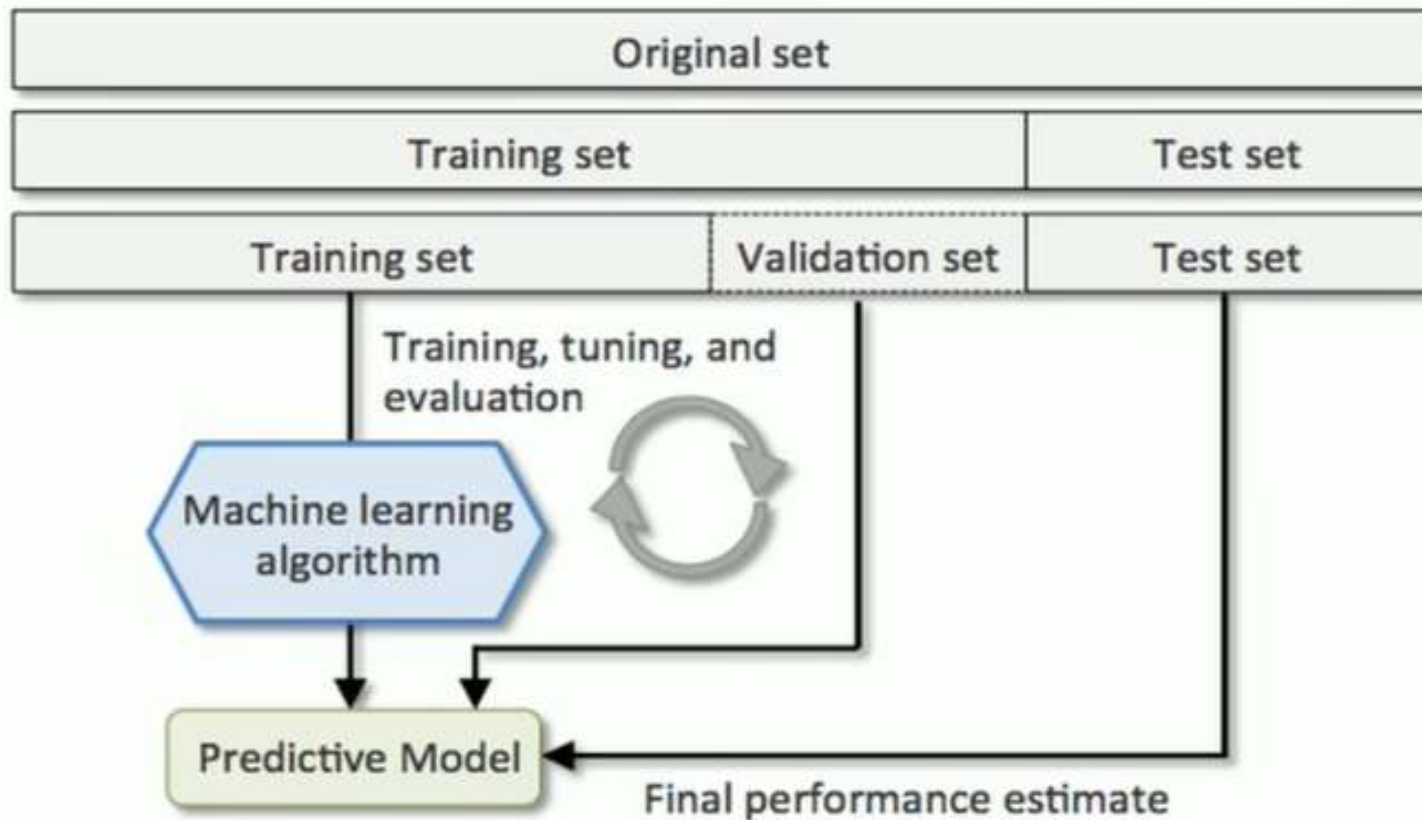
Feature selection methods allow us to:

- Reduce overfitting as less redundant data means less chance to make decisions based on noise;
- Improve accuracy by removing misleading and unimportant data;
- Reduce training time since data with fewer columns mean faster training.
- However, feature selection methods are hard to apply to high-dimensional data.
- The more features we have, the longer it takes for selection to complete.
- Also, there's the risk of overfitting when there aren't enough observations.

Data processing: Splitting datasets

When developing a machine learning model, one of the fundamental steps is to split your data into different subsets. These subsets are typically referred to as train, test, and validation data.





Train Data

Training data is the subset of the dataset used to train the model. This is where the **model learns patterns, relationships, and features of the data**. During training, the algorithm adjusts its parameters based on this data to minimize error and improve its predictions.

- **Purpose:** To fit the model.
- **Process:** The model iteratively adjusts its parameters to better fit the training data, using algorithms like gradient descent.
- **Example:** If you're building a model to predict house prices, the training data might include numerous examples of houses with known prices, along with their features like size, location, and number of bedrooms.

Validation Data

Validation data is used **to tune the model's hyperparameters and to provide an unbiased evaluation of the model** while tuning. Hyperparameters are the aspects of the model that are not learned from the data but are **set before the training process begins** (e.g., learning rate, number of layers in a neural network).

- **Purpose:** To **evaluate the model during training** and assist in hyperparameter tuning.
- **Process:** After each training iteration (or epoch), the model's performance is evaluated on the validation set. This helps to prevent overfitting to the training data.
- **Example:** In the house price prediction example, the validation data would be a separate subset of house examples that are not seen by the model during training but are used to check the model's performance and tune hyperparameters.

Test Data



Test data is the subset of the dataset used to provide an unbiased **evaluation of a final model** fit on the training dataset. It is only used after the model has been trained (and validated).

- **Purpose:** To assess the final model's performance and generalization to unseen data.
- **Process:** Once the model has been trained and hyperparameters tuned, the test data is used to evaluate the model's performance. This step is crucial to understand how the model will perform in the real world.
- **Example:** For the house price prediction model, the test data would be another separate subset of houses that the model hasn't seen during training or validation, and their known prices would be compared against the model's predictions

- 
- **The training set teaches the model, the validation set tunes it, and the test set delivers the final unbiased performance verdict.**
 - **A common starting split is 80% training, 10% validation, and 10% test — adjust based on dataset size and use case.**
 - **Reusing validation data as test data causes overfitting; keeping the test set completely unseen preserves evaluation integrity.**
 - **K-fold cross-validation maximizes limited datasets by rotating which portion serves as the validation split across multiple runs.**

Data Part	Typical Percentage	Primary Purpose
Training Set	70% - 80%	Algorithm learning and weight optimization
Testing Set	20% - 30%	Final model evaluation and accuracy metrics
Validation Set	Optional	Hyperparameter tuning and architectural changes



Why Split Data?

- 1. Prevent Overfitting:** By splitting the data, you ensure that the model isn't just memorizing the training data but is able to generalize well to unseen data.
- 2. Unbiased Evaluation:** Using a separate test set provides an unbiased evaluation of the model's performance.
- 3. Hyperparameter Tuning:** Validation sets help in tuning the model's hyperparameters to improve performance.

Cross Validation

Cross-validation is a crucial technique in machine learning used **to evaluate the performance of a model**. It helps ensure that the model generalizes well to unseen data. There are several types of cross-validation, each with its own methodology and use cases.

- The idea of cross-validation arises because of the *problems* with train test model or train test validation model.
- It basically wants to guarantee that the score of our model does not depend on the way we picked the train and test set.

Following are the type of Cross Validation Techniques

- K-folds ✓

- Stratified K-folds
- Leave-one-out
- Leave-p-out

K- Fold Cross Validation

In K-Fold Cross-Validation, the dataset is divided into K equally-sized folds. The model is trained on K-1 folds and validated on the remaining fold. This process is repeated K times, with each fold used exactly once as the validation set. The final performance metric is the average of the K validation scores.

Advantages:

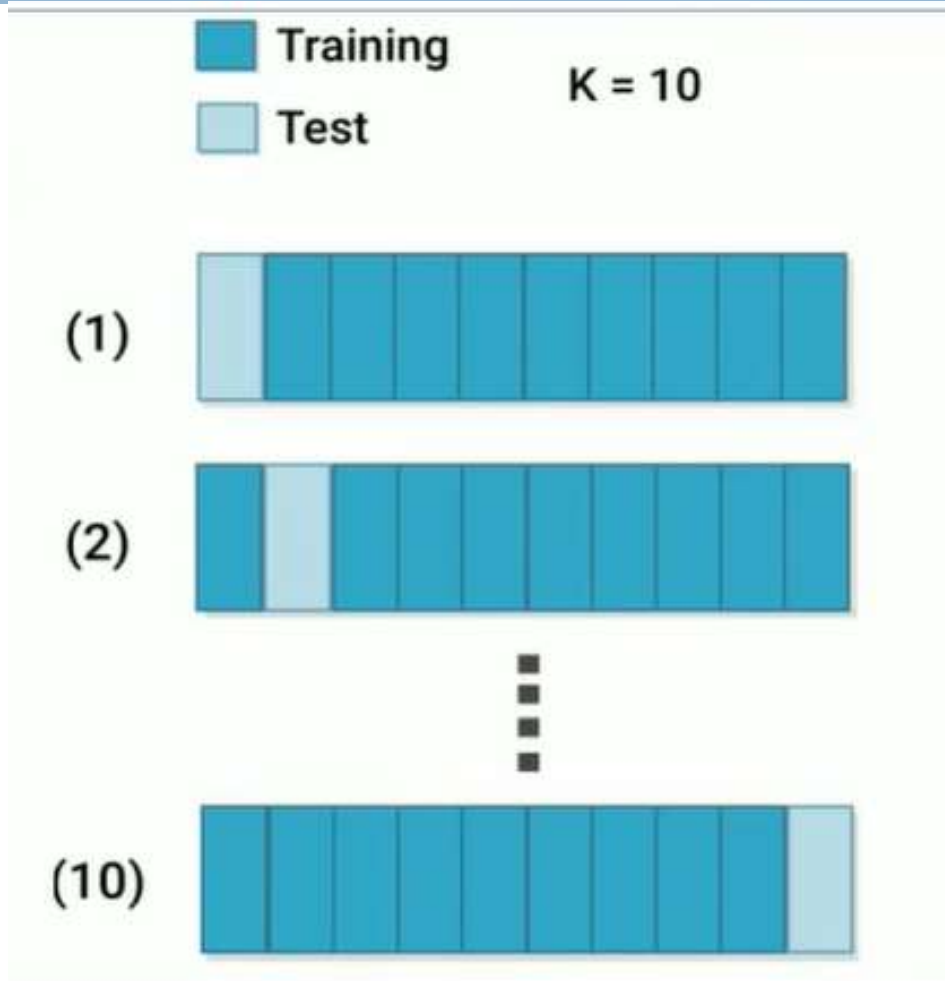
- More reliable estimate of model performance.
- Reduces variance compared to a single holdout set.

Disadvantages:

- Computationally more expensive than holdout validation.
- Not suitable for time-series data without modification.
- Not suitable for imbalanced datasets

K- Fold Cross Validation

- Entire dataset into divided into 10 folds
- Train model 10 times
- In each iteration 1 fold is considered for testing and remaining 9 for training
 - first iteration- fold 1 for testing
 - second iteration- fold 2 for testing & so on
- Take the avg-> final performance



Overfitting, Underfitting

When we're working on a machine learning project, we often face situations where we are encountering unexpected performance or error rate differences between the training set and the test set (as shown).

```
1 from sklearn.metrics import classification_report
2 print(classification_report(y_train, predicted_values))
```

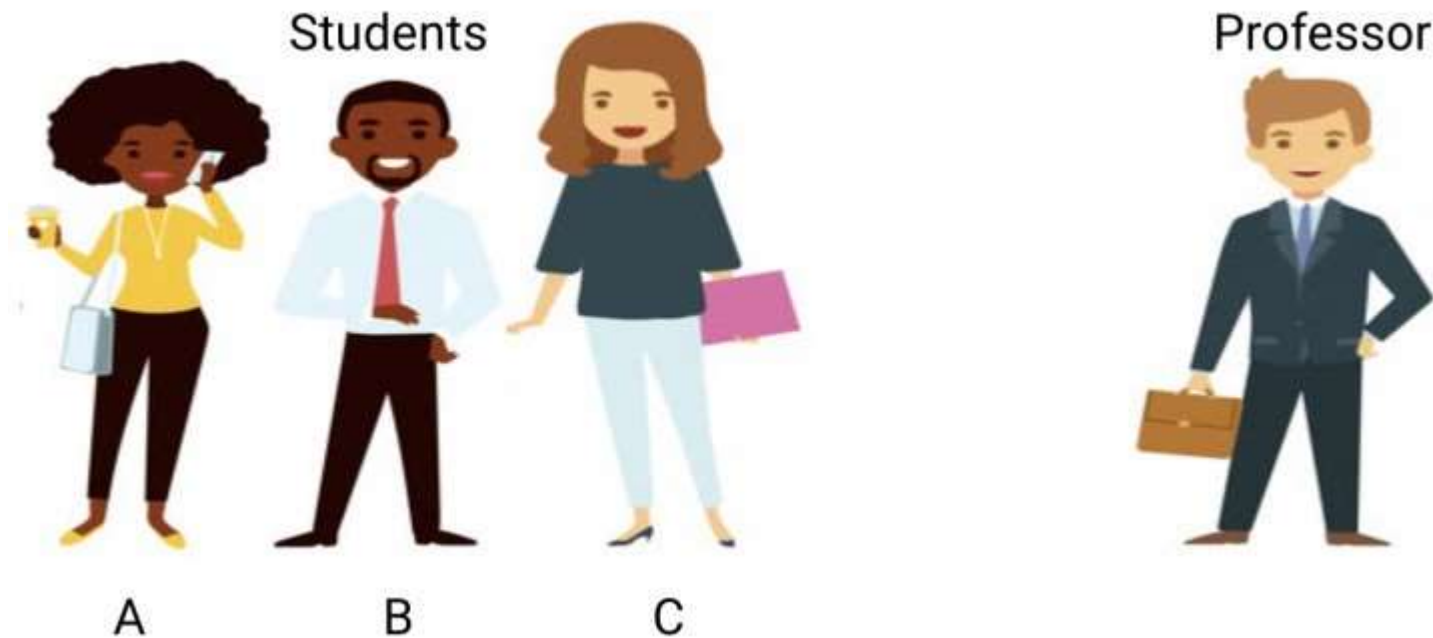
	precision	recall	f1-score	support
0	1.00	1.00	1.00	14234
1	1.00	1.00	1.00	3419
accuracy			1.00	17653
macro avg	1.00	1.00	1.00	17653
weighted avg	1.00	1.00	1.00	17653

```
1 predicted_values = classifier.predict(x_test)
2 print(classification_report(y_test, predicted_values))
```

	precision	recall	f1-score	support
0	0.87	0.86	0.87	3559
1	0.44	0.46	0.45	855
accuracy			0.78	4414
macro avg	0.66	0.66	0.66	4414
weighted avg	0.79	0.78	0.79	4414

Example

- Consider a math class consisting of 3 students and a professor.



Example

- Now, in any classroom, we can broadly divide the students into 3 categories. We'll talk about them one-by-one.



- Hobby = chatting
- Not interested in class
- Doesn't pay much attention to professor

- Let's say that student A resembles a student who does not like math. She is not interested in what is being taught in the class and therefore does not pay much attention to the professor and the content he is teaching.

Example

- Let's consider student B. He is the most competitive student who focuses on memorizing each and every question being taught in class instead of focusing on the key concepts. Basically, he isn't interested in learning the problem-solving approach.



B

- Hobby = to be best in class.
- Mugs up everything professor says.
- Too much attention to the class work.

Example

- Finally, we have the ideal student C. She is purely interested in learning the key concepts and the problem-solving approach in the math class rather than just memorizing the solutions presented.

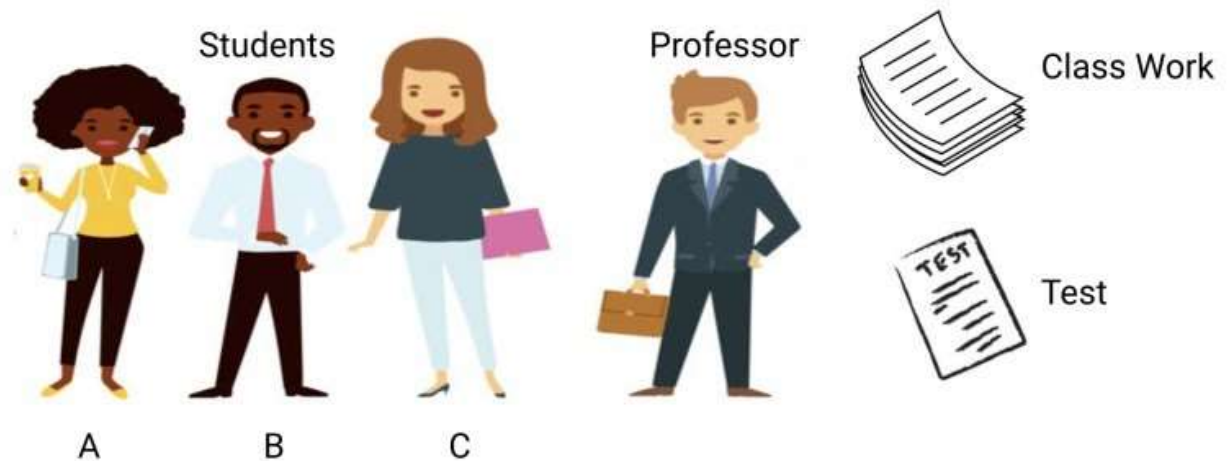


C

- Hobby = learning new things
- Eager to learn concepts.
- Pays attention to class and learns the idea behind solving a problem.

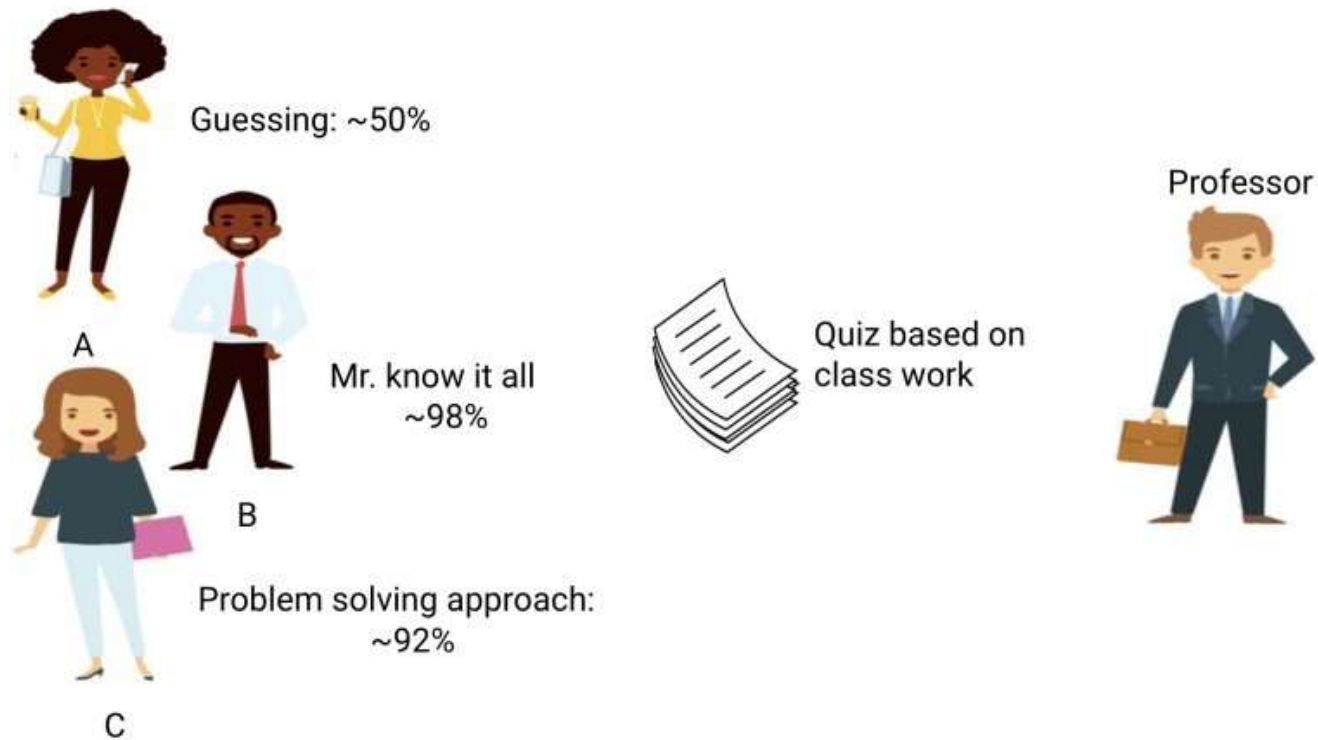
Example

- We all know from experience what happens in a classroom. The professor first delivers lectures and teaches the students about the problems and how to solve them. At the end of the day, the professor simply takes a quiz based on what he taught in the class.



Example

- So, let's discuss what happens when the teacher takes a classroom test at the end of the day:

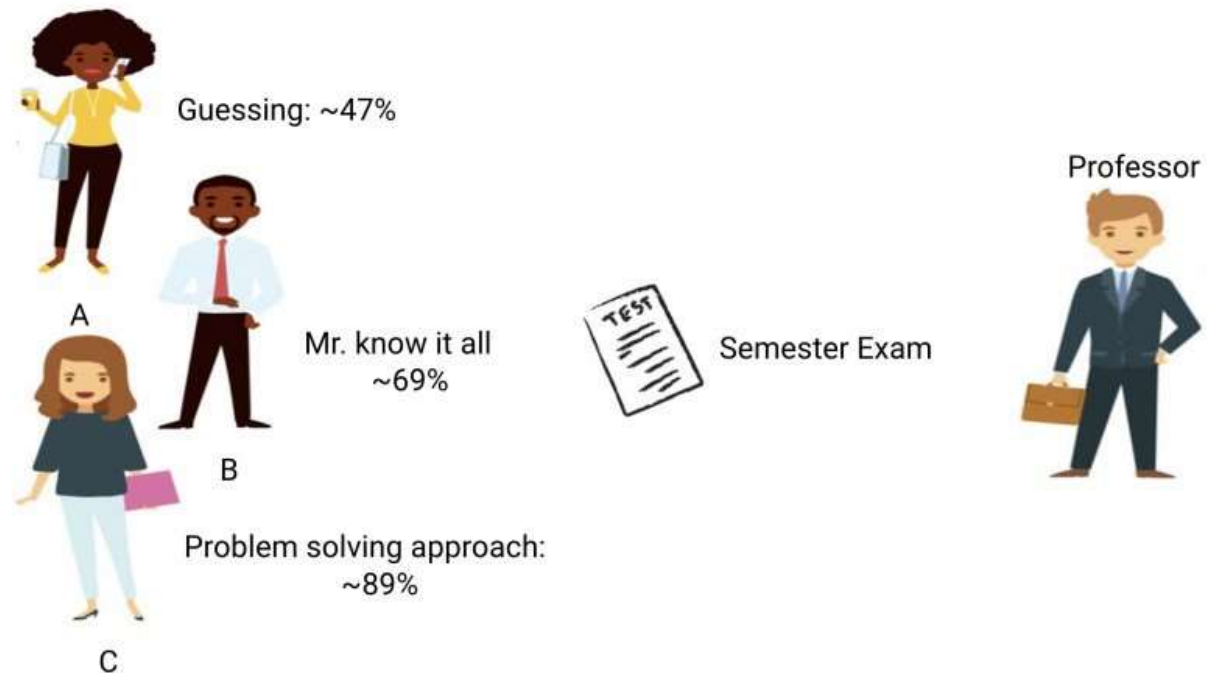


Example

- Student A, who was distracted in his own world, simply guessed the answers and got approximately 50% marks in the test
- On the other hand, the student who memorized each and every question taught in the classroom was able to answer almost every question by memory and therefore obtained 98% marks in the class test
- For student C, she actually solved all the questions using the problem-solving approach she learned in the classroom and scored 92%

Example

- Now here's the twist. Let's also look at what happens during the monthly test, when students have to face new unknown questions which are not taught in the class by the teacher.



Example

- In the case of student A, things did not change much and he still randomly answers questions correctly ~50% of the time.
- In the case of Student B, his score dropped significantly. Can you guess why? This is because he always memorized the problems that were taught in the class but this monthly test contained questions which he has never seen before. Therefore, his performance went down significantly
- In the case of Student C, the score remained more or less the same. This is because she focused on learning the problem-solving approach and therefore was able to apply the concepts she learned to solve the unknown questions.

Example



A

Not interested in learning

Class test ~50%

Test ~47%



B

Memorizing the lessons

Class test ~98%

Test ~69%



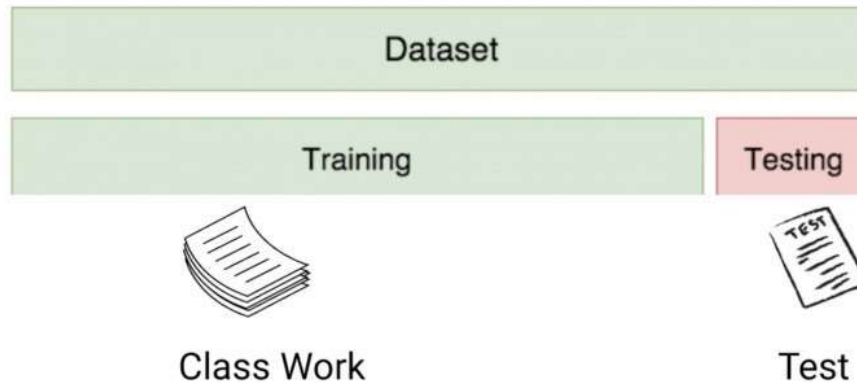
C

Conceptual Learning

Class test ~92%

Test ~89%

Example



It gave a perfect score over the training set but struggled with the test set. Comparing that to the student examples we just discussed, the classifier establishes an analogy with student B who tried to memorize each and every question in the training set.

Conclusion



A

Not interested in learning

Class test ~50%
Test ~47%

Under-fit/ biased learning



B

Memorizing the lessons

Class test ~98%
Test ~69%

Over-fit/ Memorizing



C

Conceptual Learning

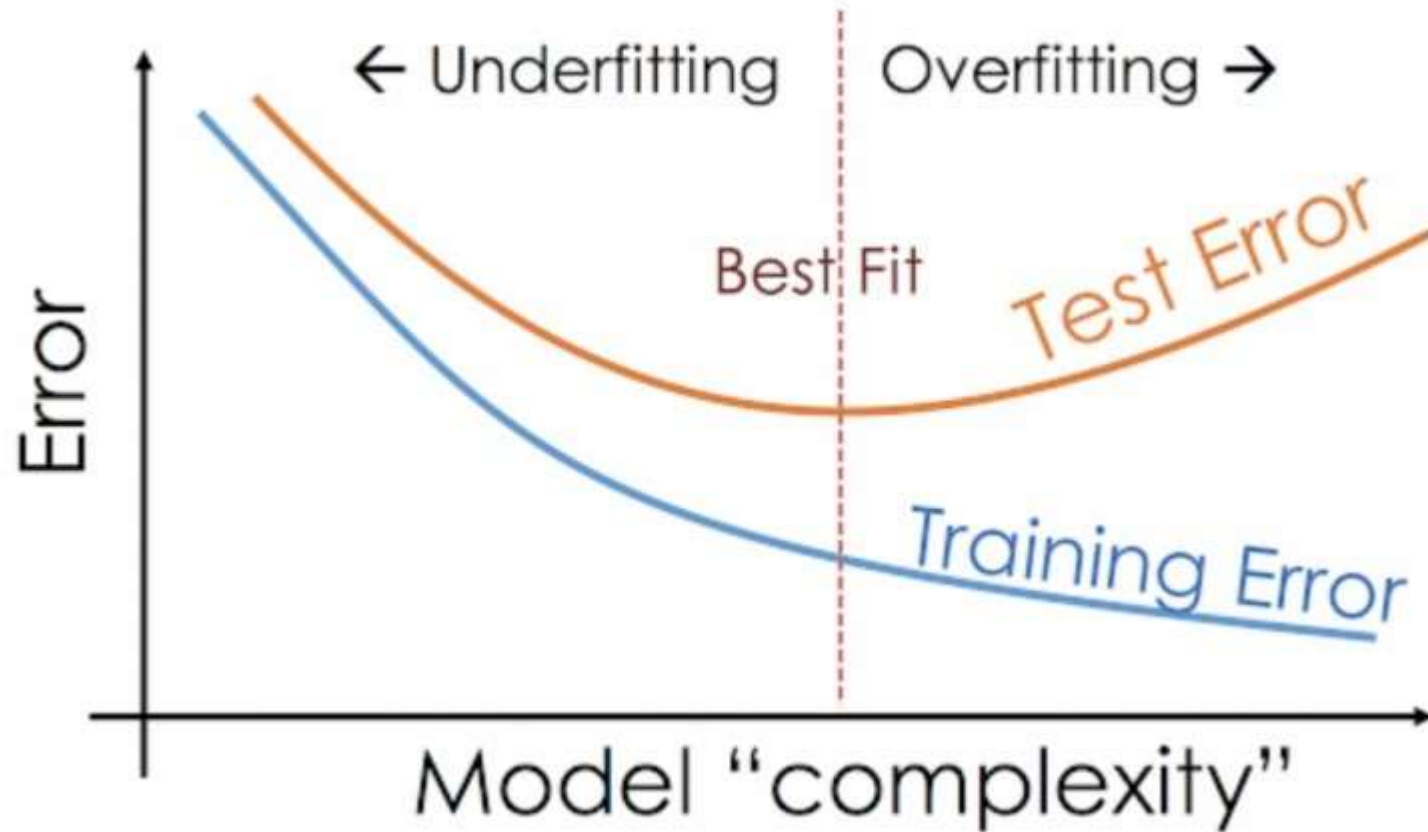
Class test ~92%
Test ~89%

Best-fit

Overfitting

- This situation where any given model is performing too well on the training data but the performance drops significantly over the test set is called an overfitting model.
- For example, non-parametric models like decision trees, KNN, and other tree-based algorithms are very prone to overfitting. These models can learn very complex relations which can result in overfitting.

Concept



Underfitting

- On the other hand, if the model is performing poorly over the test and the train set, then we call that an underfitting model.
- An example of this situation would be building a linear regression model over non-linear data.

Overfitting

- Overfitting refers to a model that models the training data too well.
- Overfitting happens when a model learns the detail and noise in the training data to the extent that it negatively impacts the performance of the model on new data.
- This means that the **noise or random fluctuations in the training data is picked up and learned as concepts** by the model.
- The problem is that these concepts do not apply to new data and negatively impact the models ability to generalize.

Underfitting

- Underfitting refers to a model that can neither model the training data nor generalize to new data.
- An underfit machine learning model is not a suitable model and will be obvious as it will have poor performance on the training data.
- Underfitting is often not discussed as it is easy to detect given a good performance metric. The remedy is to move on and try alternate machine learning algorithms.
- Nevertheless, it does provide a good contrast to the problem of overfitting.

A good fit

- Ideally, you want to select a model at the sweet spot between underfitting and overfitting.
- This is the goal, but is very difficult to do in practice.
- To understand this goal, we can look at the performance of a machine learning algorithm over time as it is learning a training data.
- We can plot both the skill on the training data and the skill on a test dataset we have held back from the training process.

Limiting overfitting

- There are two important techniques that you can use when evaluating machine learning algorithms to limit overfitting:
 - Use a resampling technique to estimate model accuracy.
 - Hold back a validation dataset.

Bias-Variance

Bias and variance are two key concepts that explain the errors a machine learning model can make during prediction.

- **Bias:** Error caused by overly simple assumptions in the model, which may lead to underfitting.
- **Variance:** Error caused by the model being too sensitive to training data, which may lead to overfitting.
- **Goal:** Balance bias and variance so the model captures patterns while still generalizing well to new data.

Bias Variance Tradeoff

The bias variance tradeoff describes the **balance between a model being too simple and too complex**.

A simple model may miss important patterns (high bias), while a very complex model may learn noise from training data (high variance).

The aim is to balance both so the model performs well on new data.

- Simple models usually have **high bias and low variance**, which may cause underfitting.
- Complex models usually have **low bias but high variance**, which may cause overfitting.
- Balanced model achieves an optimal point where both bias and variance are reasonably low.

- Goal of machine learning is to **minimize the total prediction error** on unseen data.

The total prediction error can be expressed as:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Here:

- **Bias²:** Error caused by incorrect assumptions in the model.
- **Variance:** Error caused by sensitivity to training data.
- **Irreducible Error:** Random noise in the data that cannot be eliminated.

Underfitting mainly occurs due to high bias:

- High bias means model makes strong assumptions
- Ignores patterns
- Learns an overly simple representation
- Variance is low because the model gives similar outputs even if the data changes

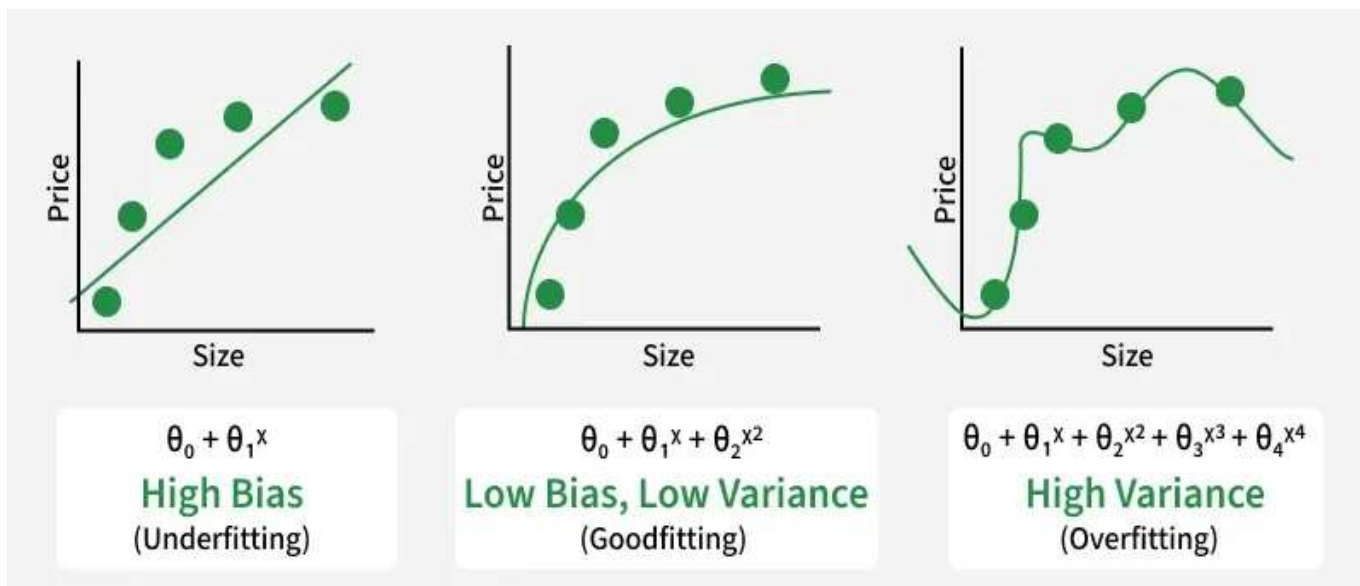
Underfitting = High Bias + Low Variance

Overfitting is mainly caused by high variance:

- High variance means model reacts too strongly to training data
- Learns noise as patterns
- Low bias because the model is extremely flexible

Overfitting = Low Bias + High Variance

- **Underfitting (High Bias):** A model that is too simple (like a straight line for curved data) misses key patterns and performs poorly on both training and testing data.
- **Overfitting (High Variance):** A model that is too complex (like a high-degree polynomial) learns noise, fits training data too closely, and performs poorly on new data.
- **Ideal Fit (Balanced):** A moderately complex model captures the main trend without following noise, giving good performance on both training and testing data.



Techniques to Reduce Underfitting

- Use a more complex model
- Add new features and perform feature engineering
- Reduce regularization
- Train for more epochs
- Scale features properly

Techniques to Reduce Overfitting

- Collect more training data
- Reduce model complexity
- Use regularization (L1/L2)
- Apply dropout (for neural networks)
- Use early stopping

- Clean noisy data

Importance of Bias Variance Tradeoff

- Helps models perform well on unseen data.
- Helps choose appropriate algorithms and complexity.
- Helps achieve better model performance.

Error Metrics in Regression Problems

Regression is a supervised learning technique used to model and analyze the relationship between input variables (features) and a continuous output variable (target). The primary objective of a regression model is to make accurate **numerical predictions**.

- They quantify the prediction error of regression models
- Different metrics emphasize different error characteristics
- Some metrics penalize large errors more than small ones
- Metric selection depends on problem requirements and data nature
- They help in model selection and optimization

Types of Regression Metrics

1. Mean Absolute Error (MAE)

Mean Absolute Error measures the average absolute difference between actual and predicted values. It treats all errors equally, regardless of their direction and provides results in the same unit as the target variable, making it easy to interpret.

Useful when all errors should be treated equally.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

n = total number of observations

y_i = actual value

$\hat{y}_i$ = predicted value

2. Mean Squared Error (MSE)

Mean Squared Error calculates the average of squared differences between actual and predicted values. By squaring errors, it penalizes larger mistakes more strongly, making it sensitive to outliers

Used as loss function during model training

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where:

- n = number of observations
- y_i = actual value
- $\hat{y}_i$ = predicted value

3. Root Mean Squared Error (RMSE)

Root Mean Squared Error is the **square root of MSE**. It maintains the strong penalty for large errors while converting the result back to the original unit of the target variable, improving interpretability.

more importance to large errors

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Where:

- n = number of observations
- y_i = actual value
- $\hat{y}_i$ = predicted value

4. R-squared (R^2 Score)

R-squared represents the **proportion of variance in the target variable** that is explained by the regression model. It provides insight into how well the model captures underlying data patterns.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Where:

- $\bar{y}$ = mean of actual values
- Numerator = residual sum of squares
- Denominator = total sum of squares

5. Mean Absolute Percentage Error (MAPE)

Mean Absolute Percentage Error expresses prediction error as a percentage of the actual values.

Easy to explain to non technical users

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Where,

- n : total number of observations
- y_i : actual value
- $\hat{y}_i$: predicted value

Metric	When to Use
MAE	You want the average size of the prediction error and want an easy-to-understand value
MSE	Large errors are particularly important and should be penalized strongly
RMSE	Large errors matter, but you want the result in the same unit as the target
R ²	You want to know how well the model explains the variation in the target
MAPE	You want to express prediction error as a percentage

Error Measurement in Classification Problems

Confusion matrix is a simple table used to measure how well a classification model is performing.

It compares the predictions made by the model with the actual results and shows where the model was right or wrong.

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

Metrics based on Confusion Matrix Data

1. Accuracy

Accuracy shows how many predictions the model got right out of all the predictions.

It gives idea of overall performance but it can be misleading when one class is more dominant over the other.

For example a model that predicts the majority class correctly most of the time might have high accuracy but still fail to capture important details about other classes.

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

2. Precision

Precision focus on the quality of the model's positive predictions.

It tells us **how many of the "positive" predictions were actually correct.**

It is important in **situations where false positives need to be minimized** such as detecting spam emails or fraud. The formula of precision is:

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

$$\text{Precision} = \frac{TP}{TP+FP}$$

3. Recall

Recall measures how good the model is at predicting positives.

It shows the **proportion of true positives detected out of all the actual positive** instances.

High recall is essential **when missing positive cases has significant consequences** like in medical tests.

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

$$\text{Recall} = \frac{TP}{TP + FN}$$

4. F1-Score

F1-score combines precision and recall into a single metric to balance their trade-off.

It provides a better sense of a model's overall performance particularly for imbalanced datasets.

It is helpful **when both false positives and false negatives are important** though it assumes precision and recall are equally important but in some situations one might matter more than the other.

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

5. Specificity

Specificity is another important metric particularly in binary classification.

It measures the **ability of a model to correctly identify negative instances**.

Specificity is also known as the True Negative Rate

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

$$\text{Specificity} = \frac{TN}{TN+FP}$$

6. Type 1 and Type 2 error

Type 1 error: It occurs when the model **incorrectly predicts a positive instance but the actual instance is negative**. This is also known as a **false positive**. Type 1 Errors affect the **precision** of a model which measures the accuracy of positive predictions.

Type 2 error: This occurs when the model **fails to predict a positive instance even though it is actually positive**. This is also known as a **false negative**. Type 2 Errors impact the **recall** of a model which measures how well the model identifies all actual positive cases.

$$\text{Type 1 Error} = \frac{FP}{FP+TN}$$

$$\text{Type 2 Error} = \frac{FN}{TP+FN}$$

Example: A diagnostic test is used to detect a particular disease in patients.

- **Type 1 Error (False Positive):** This occurs when the test predicts a patient has the disease (positive result) but the patient is actually healthy (negative case).
- **Type 2 Error (False Negative):** This occurs when the test predicts the patient is healthy (negative result) but the patient actually has the disease (positive case).

Example: Confusion Matrix for Dog Image Recognition with Numbers

Index	1	2	3	4	5	6	7	8	9	10
Actual	Dog	Dog	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Not Dog	Not Dog
Predicted	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Dog	Dog	Not Dog	Not Dog
Result	TP	FN	TP	TN	TP	FP	TP	TP	TN	TN

- Actual Dog Counts = 6
- Actual Not Dog Counts = 4
- True Positive Counts = 5
- False Positive Counts = 1
- True Negative Counts = 3
- False Negative Counts = 1

		Predicted	
		Dog	Not Dog
Actual	Dog	True Positive (TP =5)	False Negative (FN =1)
	Not Dog	False Positive (FP=1)	True Negative (TN=3)

Confusion Matrix

Machine Learning

Predicted & Actual values

		Predicted		
		No	Yes	
Actual	No	65 TN	5 FP	70
	Yes	10 FN	90 TP	100
		75	95	170

$$\text{Accuracy} = \frac{TP+TN}{\text{Total}} = \frac{90+65}{170} = 0.91$$

$$\text{Error rate} = \frac{FP+FN}{\text{Total}} = \frac{5+10}{170} = 0.09$$

$$\text{Precision} = \frac{TP}{\text{Predicted Yes}} = \frac{90}{95} = 0.95$$

$$\text{Recall} = \frac{TP}{\text{Actual Yes}} = \frac{90}{100} = 0.9$$